CAFE: A Compound-AI Factorial Evaluation Framework

Fabian Lukassen[1] **Christoph Weisser**[2] **Thomas Kneib**[1] **Alexander Silbersdorff**[1]
[1]University of Göttingen [2]Bielefeld University of Applied Sciences and Arts (HSBI)
fabian.lukassen@uni-goettingen.de christoph.weisser@hsbi.de
tkneib@uni-goettingen.de asilbersdorff@uni-goettingen.de

Abstract

We introduce **CAFE** (**C**ompound-**AI** **F**actorial **E**valuation), an open-source platform that brings *design of experiments* to the evaluation of compound AI systems (CAIS). Such systems expose many interchangeable choices – e.g. which retriever, model, or prompt – and practitioners rarely know which of them most affects answer quality. With CAFE, a practitioner registers each swappable component of a pipeline as a factor to build a factorial design over the chosen factors, run the resulting configurations, and score the answers on a shared rubric using a configurable LLM judge together with human raters. From these ratings it attributes answer-quality variance to the components and their interactions with mixed-effects models and reports effect sizes, significance, the best configuration, cost and latency trade-offs, and judge–human reliability. Whereas existing tools mostly either *search* for a good configuration or *score* outputs in isolation, CAFE also *explains* which component drives quality and whether an observed difference is significant. We validate CAFE on a retrieval-augmented question-answering (QA) pipeline over the HotpotQA benchmark dataset, where it recovers planted factor effects and stays calibrated under a permutation null. CAFE is released as a Python package and as a Web application.

1 Introduction

Modern AI applications rarely rest on a single language model. The current predominant pattern is the *compound AI system* (Zaharia et al., 2024; Chen et al., 2025a): an application that solves a task with several interacting components such as retrieval, reranking, context assembly, prompting, one or more model calls, tool use, and guardrails. Each component can be realized by several competing techniques, retrieval alone might be sparse keyword search, dense-embedding retrieval, or a reranking pipeline; any other component can be swapped just as freely. Because these discrete, often non-differentiable choices combine, even a modest pipeline induces a large, combinatorial space of candidate *configurations*. The growing literature helps practitioners *build* and *optimize* such pipelines, from declarative programming models (Khattab et al., 2024) to end-to-end configuration search (Kartal et al., 2025; Lee et al., 2025).

However, choosing which configuration to ship raises questions that optimizers only answer indirectly. Suppose that the builder of a retrieval-augmented assistant can pick among three base models, two retrieval corpora, and three prompts: 18 candidate configurations. The practitioner's immediate goal is the single best-performing one, which CAFE reports directly; but a trustworthy, actionable decision rests on two coupled questions. First, *which component drives answer quality and where is effort best spent?* A team's time is finite, and it matters whether the next gain will come from engineering the prompt or from building a stronger retrieval pipeline. Components also interact, often non-monotonically – a stronger model may help only when retrieval is good, and adding model calls can help some queries while hurting others (Chen et al., 2024) – so a disjoint perspective risks to mismeasure the system by neglecting such interactions. Second, *is an observed difference significant?* A gap between two configurations can be an artifact of chance rather than a genuine effect, and comparing averaged scores from a single run of each gives no way to tell the two apart. An optimizer returns the best-scoring configuration, but not a calibrated account of *why* it wins or *whether* the margin is trustworthy.

Landing page: https://cafe-ai.de
Demo: https://cafe-ai.de/demo
Code: https://github.com/fabian-lu/Cafe
Documentation: https://fabian-lu.github.io/Cafe

To address these issues CAFE employs *design of experiments* (DoE) and mixed-effects models (Fisher, 1935; Montgomery, 2017; Pinheiro and Bates, 2000). A factorial design estimates each component's effect *and* the interactions among components from a balanced set of runs; replicating runs and modeling the outcome with random effects separates real differences both from noise and from the fact that some inputs are simply harder than others. However, there is currently no usable system that packages this methodology for people who build compound AI pipelines.

CAFE is an open-source evaluation framework with four contributions:

(i) a **pipeline-as-factors abstraction** that surfaces any swappable component as a *factor* whose alternative implementations become its *levels*;
(ii) a **design-and-execution engine** that builds a factorial design over the chosen factors and runs each configuration against the target system;
(iii) a **dual-rating workflow** in which the same answers are scored on a shared rubric by a fully configurable LLM judge and by humans, with inter-rater reliability reported between them;
(iv) a **variance-attribution layer** that fits mixed-effects models matched to the rating scale, reports how much each component and interaction contributes with effect sizes and significance, and identifies the best configuration.

CAFE is released as an open-source early-stage Python package (licensed under Apache 2.0, installable with `pip install cafe-core`) with a companion web application (a screenshot of the web interface is in Appendix A); we also provide example notebooks for system demonstration. The project welcomes any contributions, where roadmap lists planned features, a selection of which we also highlight in Section 6.

2 Related Work

The evaluation of CAIS touches on three lines of work: tools that *optimize* pipelines, tools that *score* their outputs, and the statistics of AI evaluation. Table 1 contrasts CAFE with representative evaluation systems.

System	Pipeline level	DoE / variance attrib.	Human + judge	Replication	Config. search
RAGAS	part	–	–	–	–
ARES	part	–	–	–	–
DeepEval	✓	–	part	–	–
TruLens	✓	–	part	–	–
LangSmith	✓	–	✓	–	–
Inspect AI	✓	–	part	part	–
DSPy	✓	–	–	–	✓
RAGSmith	✓	–	–	–	✓
CAFE	✓	✓	✓	✓	–

Table 1: CAFE versus representative systems.

Optimizing compound systems. One line of work *searches* the configuration space for a high-scoring pipeline: DSPy compiles and tunes declarative pipelines (Khattab et al., 2024), TextGrad backpropagates textual feedback (Yuksekgonul et al., 2025), LLMSelector allocates models to modules (Chen et al., 2025b), Optimas aligns local rewards with global performance (Wu et al., 2026), and RAGSmith and multi-objective search explore large RAG spaces (Kartal et al., 2025; Barker et al., 2025).

Evaluation frameworks and tools. Holistic benchmarks such as HELM (Liang et al., 2023) characterise *models* across scenarios rather than *pipelines* across component choices. Metric libraries score pipeline outputs with LLM judges: RAGAS (Es et al., 2024) and ARES (Saad-Falcon et al., 2024), which adds prediction-powered confidence intervals (Fisch et al., 2024), alongside DeepEval (Confident AI, 2024), a `pytest`-style framework offering G-Eval (Liu et al., 2023) and component-level metrics, and TruLens (TruEra, 2024), which computes feedback functions over execution traces. Execution and annotation platforms (LangSmith (LangChain, 2024), Inspect AI (UK AI Security Institute, 2024), and Argilla (Argilla, 2024)) supply run harnesses, judge scoring, and human annotation queues.

Statistics for AI evaluation. A growing literature views evaluation through the lense of statistical inference: quantifying benchmark uncertainty (Wang et al., 2025), hierarchical Bayesian modeling (Luettgau et al., 2025), measurement error in LLM pipelines (Messing, 2026), and ordinal rather than interval models for rubric ratings (Howcroft and Rieser, 2021; Taylor et al., 2023; Syiem and Velloso, 2026); variance-based

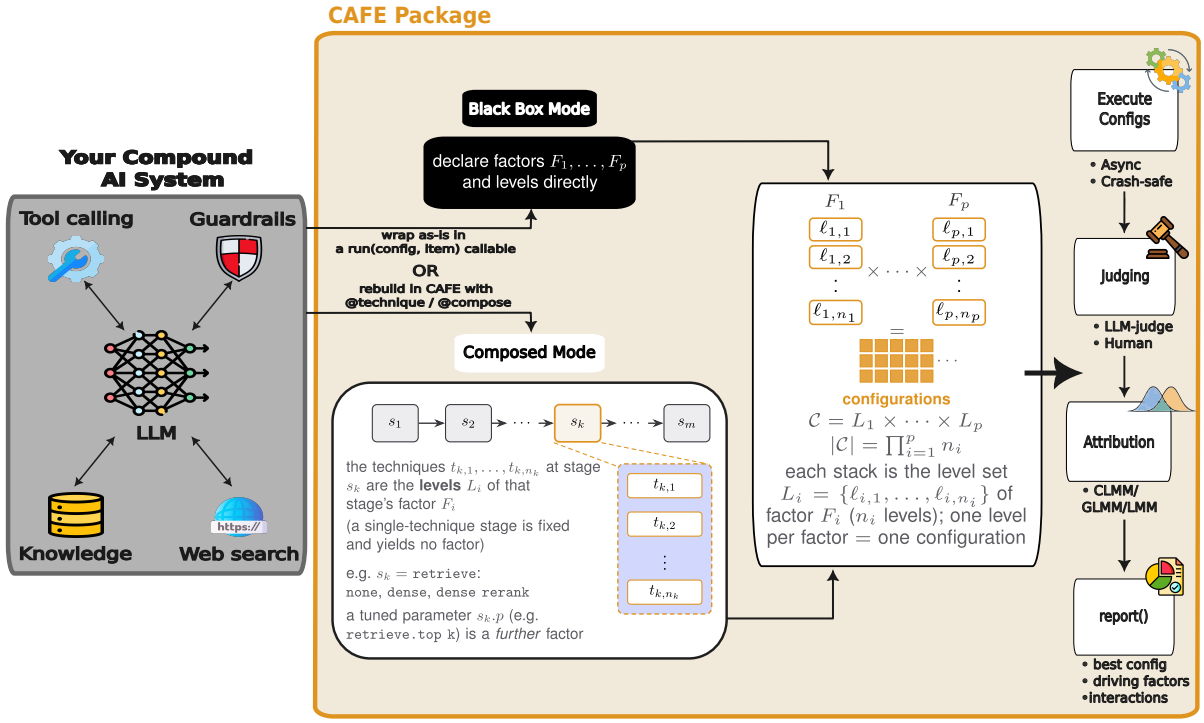


Figure 1: CAFE architecture.

sensitivity analysis is the classical instrument for apportioning output variance to inputs (Saltelli et al., 2008). The two closest works are Haase et al. (2026), who partition output variance into prompt, model and sampling components but for standalone creative generation with a linear model and a single automatic rater, and Mustahsan et al. (2025), who summarise run-to-run inconsistency with a single intraclass correlation; neither attributes variance across a designed factorial spanning the full pipeline. Recent work also attributes faults or perturbations within compound systems (Chowdhury and D'Souza, 2026; Nilayam and Nayak, 2026), while a parallel effort calibrates and audits LLM judges (Gu et al., 2024; Han et al., 2025; Boyeau et al., 2025; Park et al., 2025; Dubois et al., 2025; Guerdan et al., 2025) and documents the nondeterminism that motivates replication (Atil et al., 2024; Haldar and Hockenmaier, 2025; Bjarnason et al., 2026). Factorial studies of LLM outputs are beginning to appear (Vazquez et al., 2025; Lukassen et al., 2026).

The gap. No existing system combines these threads (Table 1). Optimizers return the best-scoring configuration but not a calibrated account of *why* it is best. Metric libraries and execution platforms score outputs and gather ratings, but leave the experimental design and statistical analysis to the user. The statistical methods above are directly relevant, yet are not packaged as a usable tool. Section 3 now describes how CAFE operationalizes this gap as a framework.

3 The CAFE Framework

CAFE turns the choices in a CAIS into experimental *factors* (Figure 1). A study varies factors $F_1, \ldots, F_p$, each ranging over a finite set of *levels* L_i; choosing one level per factor gives a *configuration* $c = (c_1, \ldots, c_p) \in \mathcal{C} = L_1 \times \cdots \times L_p$, and the *full factorial* is this entire product. Running c on an input $x \in \mathcal{X}$ yields an output $\mathrm{Run}(c, x)$ that a rubric maps to a quality rating Y; the system and judge being nondeterministic, Y is a random variable, since the same c can score differently across runs. Two kinds of choice become factors: *which component* performs a step – e.g. the retriever, with levels `none`, `dense`, `dense_rerank` – and a *parameter* of a component, such as the decoding temperature.

When a pipeline is built inside CAFE, these factors are read off its structure: the system is an ordered sequence of *stages* $s_1, \ldots, s_m$ (each a point where a choice is made), and the *techniques* registered for a stage s_k are the levels of that stage's factor, while a technique's tunable parameter $s_{k.p}$ (e.g. `retrieve.top_k`) is a further factor.

Given factors and inputs, CAFE casts the questions of Section 1 as estimation problems on Y: *attribution* – how much of the variation in Y each factor F_i and each interaction explains; *selection* – the configuration $\arg\max_c \mathbb{E}[Y \mid c]$; and *significance* for both, with valid uncertainty under the run-to-run variation of Run. Algorithm 1 formalises this procedure. We describe each part in turn.

Algorithm 1 A CAFE study

1: declare factors $F_1, \ldots, F_p$ and their levels $L_1, \ldots, L_p$
2: $\mathcal{C} \leftarrow L_1 \times \cdots \times L_p$ ▷ full factorial: every configuration
3: **for all** configuration $c \in \mathcal{C}$, input $x \in \mathcal{X}$, replicate r **do**
4: $\quad a_{cxr} \leftarrow \textsc{Run}(c, x)$ ▷ execute; concurrency-bounded, resumable
5: $\quad y_{cxr} \leftarrow \textsc{Rate}(a_{cxr})$ ▷ LLM judge and/or human, shared rubric
6: **end for**
7: fit the model matched to the rubric scale (CLMM / logistic / linear)
8: attribute variance; report effect sizes, significance, the best configuration, the cost frontier, and α

3.1 Defining a system

CAFE supports two ways to supply Run from Algorithm 1. In the common case, the pipeline is *composed* in CAFE itself: each technique is registered on its stage with a one-line decorator, and an orchestration function wires the stages together (Listing 1). CAFE then varies each factor over its levels and, because it executes the pipeline, records per-stage latency and dollar cost with no extra wiring. One level of a factor may be `None`, which tells CAFE to skip that stage entirely, so a component can be switched on and off (e.g. running with or without a reranker). Alternatively, a system that cannot or need not be instrumented – e.g. an external HTTP endpoint – is driven as a *black box*: any callable `run(config, item)` is a system – CAFE passes it a configuration and an input, and the callable reads each factor's chosen level from `config` – and its factors are declared directly. Black-box mode gives up only the per-stage latency and cost breakdown, leaving the design, rating, and variance attribution unchanged.

```python
import cafe
pipe = cafe.Pipeline()

# `retrieve`: two techniques = the levels of the retrieve
    factor
@pipe.technique(stage="retrieve", name="none")
async def retrieve_none(ctx, query):
    return [] # parametric memory only

@pipe.technique(stage="retrieve", name="dense", cost_usd
    =0.0004)
async def retrieve_dense(ctx, query, top_k=8):
    return dense_search(query, k=top_k) # top_k -> a
        retrieve.top_k factor

# `generate`: the `model` keyword becomes the generate.
    model factor
@pipe.technique(stage="generate", name="llm")
async def generate(ctx, query, docs, model="gpt-oss-20b"):
    return await cafe.complete(model, prompt(query, docs))

@pipe.compose
async def rag(config, item, ctx): # wire the stages
    together
    docs = await ctx.run("retrieve", query=item["text"])
    return await ctx.run("generate", query=item["text"],
        docs=docs)

study = cafe.Study(
    name="rag-study",
    system=pipe,
    factors=[pipe.factor("retrieve"),
            cafe.Factor("generate.model", ["gpt-oss-20b", "
                gpt-oss-120b"])],
    dataset=data, rubric=rubric, judge=judge, replications
        =2,
)
```

Listing 1: Composed mode: techniques are registered on stages with decorators and `@compose` wires them into a pipeline; `cost_usd` attaches a per-call price CAFE tracks. The same `Study` runs unchanged in black-box mode over any `run(config, item)` callable.

3.2 Design, cost, and execution

From the chosen factors, CAFE enumerates a *full factorial* – every combination of levels – and `cafe.preflight` reports the configuration count, an estimated compute time and dollar cost, and a design-adequacy check (for instance, too few inputs for a stable random effect) before anything runs. Execution then runs each pair $(c, x) \in \mathcal{C} \times \mathcal{X}$ under a configurable concurrency bound and checkpoints incrementally, so a study interrupted after hours resumes rather than restarts. Every cell is run with *replication*: a compound system can return different outputs on repeated calls even at temperature zero (Atil et al., 2024), and the repeats let the analysis estimate within-cell variation instead of mistaking it for an effect. Per-stage latency and cost are recorded throughout, feeding the multi-objective analysis of Section 3.4.

3.3 Rating

Each answer is scored on a *rubric* that fixes an *ordinal*, *binary*, or *numeric* scale. Two rater types apply the same rubric. An *LLM judge* is fully configurable – its model, its prompt, and the rubric it applies are all chosen by the user, and it scores with or without a reference answer; CAFE ships prompt presets and stores the exact prompt and raw response behind every verdict for auditing. Repeated judge passes are retained so that judge self-inconsistency can be measured (Haldar and Hockenmaier, 2025). *Humans* rate through a fillable answer sheet (`cafe.answer_sheet` exports it as csv, `cafe.human_ratings` reads it back), thus

allowing reliability analysis between them.

3.4 Statistical analysis

The statistical analysis is CAFE's analytical core, reported in three layers, and it is deliberately driven by the rubric: the declared scale type selects the model, so each response is analysed on the scale it lives on rather than coerced to an interval mean, and the user never has to choose a statistical test.

Descriptive. Per-level marginal means, per-configuration means, and the best configuration give the immediate picture of the design.

Scale-matched inference. Write Y for an answer's rating and y_{cxr} for its value under configuration c on input x, replicate r, with c summarised by its factor-level design vector $\mathbf{z}_c$ (main effects and, by default, two-way interactions), and let $\eta = \mathbf{z}_c^\top \boldsymbol{\beta} + u_x$ be the linear predictor, where $u_x \sim \mathcal{N}(0, \sigma^2)$ is a per-input random intercept. The declared scale selects one of three mixed models:

$$\begin{aligned} &\text{ordinal (CLMM):} && \operatorname{logit} \Pr(Y \le q) = \theta_q - \eta, \\ &\text{binary (GLMM):} && \operatorname{logit} \Pr(Y = 1) = \eta, \\ &\text{numeric (LMM):} && Y = \eta + \varepsilon. \end{aligned}$$

The ordinal case is a *cumulative link mixed model* (McCullagh, 1980; Agresti, 2002; Taylor et al., 2023); unlike the binary and numeric models it has no fixed intercept, as its $K-1$ ordered thresholds $\theta_1 < \cdots < \theta_{K-1}$ play that role (which keeps $\mathbf{z}_c^\top \boldsymbol{\beta}$ identifiable). All three share the per-input *random* intercept u_x, which absorbs the fact that some inputs are simply harder, so factor effects are estimated *within* input. Because Python has no standard fitter for the ordinal and binary models, CAFE fits them by calling R in a subprocess, `ordinal::clmm` (Christensen, 2023) and `lme4::glmer` (Bates et al., 2015), passing the data as a table and reading the estimates. The linear model is fit natively. The scale-matched model yields one coefficient per factor level, relative to a reference level and including the interaction cells, each with its own significance test. It therefore states whether, and in which direction, a specific choice matters, but offers no single summary per factor.

Effect sizes and attribution. Alongside the scale-matched model, a linear mixed model gives an *approximate*, scale-agnostic effect-size view: a Type-II analysis of variance (Langsrud, 2003) gives each factor and each interaction an F-test and a partial η^2 (Cohen, 1988), a model R^2 (Nakagawa and Schielzeth, 2013), and pairwise contrasts give Cohen's d (Cohen, 1988). Ranking factors by partial η^2 is a variance attribution (Saltelli et al., 2008) that answers "which component drives quality," and the best end-to-end configuration is read from the fitted means. The two layers are read together: the direction and significance of a specific choice come from the scale-matched model, whereas which factor matters, and by how much, comes from η^2 and d. Because it treats the scale as interval, this view is a standardized summary for ranking and cross-study comparison rather than the primary inference (Section 6). Its random intercept is the same u_x; when that variance is estimated at zero or cannot be estimated, so that no between-input variation remains once the factors are accounted for, the mixed model coincides with the fixed-effects analysis of variance, which CAFE then reports and flags.

Reliability and robustness. Because the quality signal is itself produced by raters, CAFE computes Krippendorff's α (Krippendorff, 2004) between the LLM judge and the humans, and among the humans, with the distance metric matched to the scale (ordinal, nominal, or interval). Finally, because latency and cost are recorded, CAFE reports the *Pareto frontier* (Barker et al., 2025) over quality, cost, and latency, exposing a configuration that is marginally better, but far slower or more expensive.

3.5 Implementation and availability

CAFE is distributed as a Python package with modules for design, execution, judging, and statistics. A web application exposes the same engine over a FastAPI (JSON) API that a React front-end consumes, backed by a PostgreSQL store; studies run as in-process asynchronous task, and the whole stack is deployed with Docker Compose. The app discovers the available systems from a configured techniques directory, where `@compose` and `@technique` registrations populate its factor/level menus.

4 Case Study and Evaluation

We validate CAFE against **known answers**: we run it on a real compound system built so that certain effects hold *by construction*, then check whether CAFE recovers them (the full study is a runnable notebook). Concretely, CAFE must *recover* an effect we know is present while ignoring one we know is absent (E1, construct validity); *not invent* significance once we destroy all real struc-

ture (E2, calibration); *agree with human* raters, since the whole analysis rests on the judge's scores (E3); and select a best configuration that *generalizes* beyond the exact questions asked (E4).

The task. The target system is a retrieval-augmented QA pipeline evaluated on **HotpotQA** (Yang et al., 2018), a multi-hop question-answering benchmark in which answering a question requires combining facts from two different Wikipedia *paragraphs*. Each question comes with (i) a short gold answer and (ii) ten candidate paragraphs, of which two are the *gold* paragraphs that contain the supporting facts and eight are *distractors*. Thus, we know both the correct answer and the relevant evidence. We keep the hard, multi-hop, non-yes / no questions (yes / no answers are guessable and would blunt the retrieval signal) and sample $n = 50$.

Screening. Because a question's own ten paragraphs make retrieval trivial, we pool *every* question's paragraphs into one shared ∼500-document corpus, embedded with `bge-m3`: each question's two gold paragraphs must now be found among ∼500. Before spending anything on the full study we *screen* this corpus by measuring *gold-paragraph recall@k* – how often the retriever's top-k actually contains a question's gold paragraphs. Recall is 0.76 at k=2, 0.87 at k=4, and 0.97 at k=12. It confirms the gold evidence is retrievable, so a no-retrieval baseline is *guaranteed* to be worse, the known-signed anchor. Furthermore, it shows that plain top-2 retrieval misses the gold about a quarter of the time, while the top-12 almost always contains it: that gap is precisely the room a *reranker* would need to help.

The factors. The system exposes three factors ($3 \times 3 \times 2 = 18$ configurations):

- `retrieve` ∈ {none, dense, dense_rerank} – the **anchor**. `none` passes no evidence (the model answers from parametric memory); `dense` feeds the `bge-m3` top-2; `dense_rerank` has an LLM reorder the top-12 down to 2, a RankGPT-style reranker (Sun et al., 2023).
- `finalize` ∈ {on, off} – the **negative control**. Both levels return the answer unchanged, so the true effect is exactly zero and CAFE must call it non-significant.
- `generate.model` ∈ {gemma3-4b, gpt-oss-20b, gpt-oss-120b} – an **expected** effect: larger models should score higher (Kaplan et al., 2020; Roberts et al., 2020); open is *by how much*, and whether strong retrieval makes model size matter less.

Scoring. Each of the $18 \times 50 = 900$ answers is graded on a three-level ordinal correctness rubric – 0 incorrect/off-topic, 1 partially correct, 2 fully correct (exact wording irrelevant) – by a *separate*, strong judge model (DeepSeek-V4-Pro). The judge is shown the question, the *gold reference answer*, and the answer to grade, and is instructed to score factual correctness against the reference. The complete `report()` output of this study is in Appendix A (Listing 2).

E1: recovery of known effects. Reading Listing 2 top to bottom, CAFE reproduces what we built in. The **anchor** is recovered with the right order and dominance: the marginal mean quality rises `none` 0.75 < `dense` 1.48 < `dense_rerank` 1.71, and `retrieve` is by far the largest effect in the linear layer (partial $\eta^2 = 0.228$); on the ordinal model, `none` lowers the log-odds of a higher score by 3.94 relative to `dense` ($p < 10^{-3}$) and `dense_rerank` raises them by 1.15 ($p = 0.008$). The reranker's edge over plain `dense` is small but significant (Cohen's $d = 0.31$), as the recall screen suggested it would be. The **negative control** `finalize` is null on every layer (CLMM $p = 0.85$, and $p > 0.4$ for each of its interactions; linear layer $\eta^2 \approx 0$, $p = 0.68$), and `generate.model` behaves as expected – larger models score higher (gpt-oss-120b +1.25 log-odds over gemma3-4b, $p = 0.004$; gpt-oss-20b +0.80, $p = 0.054$; linear layer $\eta^2 = 0.052$). The best configuration is `dense_rerank` + `gpt-oss-120b` (mean 1.78), with `finalize` immaterial. Full statistics (F, Cohen's d, and CLMM log-odds) are in Listing 2. One note in that listing deserves comment: in the linear layer the between-question variance was estimated at zero, so its F-tests and effect sizes come from the fixed-effects analysis of variance described in Section 3.4. The ordinal model, fit with the same random intercept, converged and agrees on every factor, and E2 below checks directly whether the reported p-values are calibrated.

Interactions. The factorial also flags a significant `retrieve` × `generate.model` interaction ($p = 0.002$ in the linear layer). The cell means show its shape: the model gap is widest with no retrieval (`none`: gemma3-4b 0.36 vs. gpt-oss-120b 1.13) and narrows as retrieval improves (`dense_rerank`: 1.59 vs. 1.78). Equivalently, re-

trieval helps the *weaker* model most – gemma3-4b gains $+1.23$ from `none` to `dense_rerank`, gpt-oss-120b only $+0.65$ – because with the evidence in context even a small model can read the answer off, whereas with none it falls back on weaker parametric knowledge (the CLMM confirms the sign: `none` $\times$ gpt-oss-120b $= +1.85$, $p < 10^{-3}$). A one-factor-at-a-time sweep cannot see this; surfacing it is exactly what a factorial design is for.

E2: does CAFE invent effects? E2 is the converse of E1: does CAFE avoid *inventing* effects that are not there? We build data in which, by construction, *no* factor has any effect, randomly reshuffling the 900 verdicts onto different answers so that every link between a factor and quality is broken. A calibrated test should then flag factors at the nominal Type I error rate, $\alpha = 0.05$; over 500 independent reshuffles CAFE flags them 4.9% of the time – indicating no spurious significance.

E3: does the judge track human judgement? The whole analysis rests on the LLM judge's scores, so the final check is whether they agree with people. Two of the authors independently re-scored a stratified sample of 61 answers, applying the same rubric *blind* to the judge's verdicts. Measured with Krippendorff's ordinal α, the two raters agree strongly with *each other* ($\alpha = 0.91$), and the LLM-judge agrees with them about as well – $\alpha = 0.95$ and 0.89, yielding $\alpha = 0.92$ across all three. Every value lands in the conventional "reliable" band ($\alpha \geq 0.80$).

E4: does the winner survive new questions? Finally, is the best configuration a real winner or an artifact of these particular 50 questions? Using cross-validation, we repeatedly split the questions into two random halves, recompute the configuration ranking on each half independently, and measure how well the two rankings agree. Across splits the results correlate strongly (Spearman $\rho = 0.91$). Disagreement is only found among a cluster of near-tied top configurations, the very gaps CAFE had already reported as non-significant.

5 Conclusion

CAFE brings design of experiments to the evaluation of compound AI systems, turning pipeline components into factors and attributing answer-quality variance to them and their interactions with a mixed-effects model matched to the rubric's scale. On a retrieval-augmented HotpotQA study it recovers planted effects, calls a negative control null, stays calibrated under a permutation null, and agrees with human raters. CAFE is open-source, and aims to make designed, statistically grounded evaluation routine for the teams who build compound AI systems.

6 Limitations and Future Work

CAFE is an early-stage framework that packages established methodology, factorial design and mixed-effects modelling, for compound AI pipelines rather than proposing a new estimator or search procedure. CAFE's own limitations are statistical and computational. Its scale-agnostic effect-size view (partial η^2, Cohen's d) treats the ordinal scale as an interval and is only an approximate ranking aid; mixed-effects fits can be unstable on small or sparse designs, as the near-singular random effect in our study shows (Section 4); and full-factorial designs grow multiplicatively in cost. The maximum interaction order is user-configurable, from main effects only up to the full set of interactions, and the design checks warn when a design is too sparse for the requested terms, but they cannot make a small study informative. Planned extensions address these: an order-independent *deviance partition* that apportions explained deviance across factors (Grömping, 2006), together with likelihood-ratio tests of whole factors on the ordinal model so that attribution no longer relies on the approximate η^2 layer, fractional-factorial screening to cover large spaces at a fraction of the runs, and Bayesian mixed models (Bürkner, 2017) for full posterior effect sizes and credible intervals. Modelling *institutional* rater effects – how different peer groups score the same system – is a further direction (Luettgau et al., 2025).

A designed study is more expensive than a single run of each candidate. That is the price of attribution, and `cafe.preflight` makes it visible before anything runs. CAFE currently measures configurations rather than searching for them, so optimizers such as DSPy or RAGSmith are complementary: the configurations they propose can serve as the levels of a factor. We plan to bring both sides into one platform by shipping a library of ready-made techniques, building blocks from which a compound system can be assembled and configured inside CAFE, and by adding optimization over them: sequential testing with early stopping, Bayesian opti-

mization or bandit-style allocation of runs across configurations, and reuse of shared computation across cells.

One caveat concerns our *evaluation* rather than CAFE itself: it validates the instrument on a relatively easy task – a three-level (0–2) correctness judgment against a gold answer is not where LLM judges struggle, and a subjective rubric (tone, style, helpfulness) would likely show lower judge–human agreement. The evaluation is narrow in other respects as well: one benchmark, $n = 50$ questions, and a single system output per configuration and question ($18 \times 50 = 900$ answers). Replication, which Section 3 motivates, was therefore not exercised. Within-cell run-to-run variation is absorbed into the residual rather than estimated separately, and the calibration and stability evidence for this study rests on E2 and E4. Agents, tool use, multi-turn systems, and subjective quality criteria are within reach of black-box mode but have not been demonstrated. Further case studies and educational material are planned as future work. Judging is where CAFE will also grow: multi-criteria scoring (several named scores per answer), G-Eval-style scoring that weights the rubric levels by their token probabilities (Liu et al., 2023), and, where a few human labels are available, prediction-powered inference to *debias* the judge's estimates against them (Boyeau et al., 2025; Fisch et al., 2024; Park et al., 2025).

Finally, CAFE has not been evaluated with its users. The web interface has had no usability study, and the report is dense with statistical terminology. Each statistic is accompanied by a short plain-language reading and the documentation walks through a complete report, but a practitioner study, a simplified dashboard that foregrounds these readings, and smoother integration into existing compound systems are the next steps on the usability side.

References

Alan Agresti. 2002. *Categorical Data Analysis*, 2nd edition. John Wiley & Sons, New York.

Argilla. 2024. Argilla: The collaboration platform for AI engineers and domain experts. Software.

Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. 2024. Non-determinism of "deterministic" LLM settings. *arXiv preprint arXiv:2408.04667*.

Matthew Barker, Andrew Bell, Evan Thomas, James Carr, Thomas Andrews, and Umang Bhatt. 2025. Faster, cheaper, better: Multi-objective hyperparameter optimization for LLM and RAG systems. *arXiv preprint arXiv:2502.18635*.

Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1):1–48.

Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. 2026. On randomness in agentic evals. *arXiv preprint arXiv:2602.07150*.

Pierre Boyeau, Anastasios Nikolas Angelopoulos, Tianle Li, Nir Yosef, Jitendra Malik, and Michael I. Jordan. 2025. AutoEval done right: Using synthetic data for model evaluation. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 5276–5290.

Paul-Christian Bürkner. 2017. brms: An R package for Bayesian multilevel models using Stan. *Journal of Statistical Software*, 80(1):1–28.

Jiayi Chen, Junyi Ye, and Guiling Wang. 2025a. From standalone LLMs to integrated intelligence: A survey of compound AI systems. *arXiv preprint arXiv:2506.04565*.

Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Ion Stoica, Matei Zaharia, and James Zou. 2024. Are more LLM calls all you need? towards the scaling properties of compound AI systems. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Matei Zaharia, James Zou, and Ion Stoica. 2025b. Optimizing model selection for compound AI systems. *arXiv preprint arXiv:2502.14815*.

Sayak Chowdhury and Meenakshi D'Souza. 2026. SETA: Statistical fault attribution for compound AI systems. In *Proceedings of the IEEE/ACM International Conference on AI Engineering (CAIN)*.

Rune Haubo Bojesen Christensen. 2023. *ordinal—Regression Models for Ordinal Data*. R package version 2023.12-4.

Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences*, 2nd edition. Lawrence Erlbaum Associates, Hillsdale, NJ.

Confident AI. 2024. DeepEval: An open-source LLM evaluation framework. Software.

Magda Dubois, Harry Coppock, Mario Giulianelli, Timo Flesch, Lennart Luettgau, and Cozmin Ududec. 2025. Skewed score: A statistical framework to assess autograders. *arXiv preprint arXiv:2507.03772*.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, pages 150–158.

Adam Fisch, Joshua Maynez, R. Alex Hofer, Bhuwan Dhingra, Amir Globerson, and William W. Cohen. 2024. Stratified prediction-powered inference for effective hybrid evaluation of language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Ronald A. Fisher. 1935. *The Design of Experiments*. Oliver and Boyd, Edinburgh.

Ulrike Grömping. 2006. Relative importance for linear regression in R: The package relaimpo. *Journal of Statistical Software*, 17(1):1–27.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2024. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*.

Luke Guerdan, Solon Barocas, Kenneth Holstein, Hanna Wallach, Zhiwei Steven Wu, and Alexandra Chouldechova. 2025. Validating LLM-as-a-judge systems under rating indeterminacy. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*.

Jennifer Haase, Jana Gonnermann-Müller, Paul H. P. Hanel, Nicolas Leins, Thomas Kosch, Jan Mendling, and Sebastian Pokutta. 2026. Within-model vs between-prompt variability in large language models for creative tasks. *arXiv preprint arXiv:2601.21339*.

Rajarshi Haldar and Julia Hockenmaier. 2025. Rating roulette: Self-inconsistency in LLM-as-a-judge frameworks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24986–25004.

Steve Han, Gilberto Titericz Junior, Tom Balough, and Wenfei Zhou. 2025. Judge's verdict: A comprehensive analysis of LLM judge capability through human agreement. *arXiv preprint arXiv:2510.09738*.

David M. Howcroft and Verena Rieser. 2021. What happens if you treat ordinal ratings as interval data? human evaluations in NLP are even more underpowered than you think. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8932–8939.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Muhammed Yusuf Kartal, Suha Kagan Kose, Korhan Sevinç, and Burak Aktas. 2025. RAGSmith: A framework for finding the optimal composition of retrieval-augmented generation methods across datasets. *arXiv preprint arXiv:2511.01386*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Klaus Krippendorff. 2004. *Content Analysis: An Introduction to Its Methodology*, 2nd edition. Sage, Thousand Oaks, CA.

LangChain. 2024. LangSmith: A platform for LLM application evaluation and observability. Software.

Øyvind Langsrud. 2003. ANOVA for unbalanced data: Use Type II instead of Type III sums of squares. *Statistics and Computing*, 13(2):163–167.

Yu-Ang Lee, Guan-Ting Yi, Mei-Yi Liu, Jui-Chao Lu, Guan-Bo Yang, and Yun-Nung Chen. 2025. Compound AI systems optimization: A survey of methods, challenges, and future directions. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 28760–28775.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, and 31 others. 2023. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Lennart Luettgau, Harry Coppock, Magda Dubois, Christopher Summerfield, and Cozmin Ududec. 2025. HiBayES: A hierarchical bayesian modeling framework for AI evaluation statistics. *arXiv preprint arXiv:2505.05602*.

Fabian Lukassen, Jan Herrmann, Christoph Weisser, Benjamin Saefken, and Thomas Kneib. 2026. From XAI to stories: A factorial study of LLM-generated explanation quality. *arXiv preprint arXiv:2601.02224*.

Peter McCullagh. 1980. Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B (Methodological)*, 42(2):109–142.

Solomon Messing. 2026. Hidden measurement error in LLM pipelines distorts annotation, evaluation, and benchmarking. *arXiv preprint arXiv:2604.11581*.

Douglas C. Montgomery. 2017. *Design and Analysis of Experiments*, 9th edition. John Wiley & Sons, Hoboken, NJ.

Zairah Mustahsan, Abel Lim, Megna Anand, Saahil Jain, and Bryan McCann. 2025. Stochasticity in agentic evaluations: Quantifying inconsistency with intraclass correlation. *arXiv preprint arXiv:2512.06710*.

Shinichi Nakagawa and Holger Schielzeth. 2013. A general and simple method for obtaining r^2 from generalized linear mixed-effects models. *Methods in Ecology and Evolution*, 4(2):133–142.

Prashanti Nilayam and Sankalp Nayak. 2026. QUIVER: A formal framework for quantifying perturbation propagation and bifurcation in compound AI systems. *arXiv preprint arXiv:2605.23956*.

Sangwoo Park, Matteo Zecchin, and Osvaldo Simeone. 2025. Adaptive prediction-powered AutoEval with reliability and efficiency guarantees. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*.

José C. Pinheiro and Douglas M. Bates. 2000. *Mixed-Effects Models in S and S-PLUS*. Statistics and Computing. Springer, New York.

Adam Roberts, Colin Raffel, and Noam Shazeer. 2020. How much knowledge can you pack into the parameters of a language model? In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, pages 338–354.

Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. 2008. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 14918–14937.

Brandon Victor Syiem and Eduardo Velloso. 2026. Better assumptions, stronger conclusions: The case for ordinal regression in HCI. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI)*.

Jack E. Taylor, Guillaume A. Rousselet, Christoph Scheepers, and Sara C. Sereno. 2023. Rating norms should be calculated from cumulative link mixed effects models. *Behavior Research Methods*, 55(5):2175–2196.

TruEra. 2024. TruLens: Evaluation and tracing for LLM applications. Software.

UK AI Security Institute. 2024. Inspect AI: Framework for large language model evaluations. Software.

Alan R. Vazquez, Kilian M. Rother, and Marco V. Charles-Gonzalez. 2025. A systematic assessment of large language models for constructing two-level fractional factorial designs. *arXiv preprint arXiv:2512.17113*. Published in Quality Engineering (2026), DOI: 10.1080/08982112.2026.2625951.

Baiqiang Wang, Dongfang Zhao, Nathan R. Tallent, and Luanzheng Guo. 2025. On the reproducibility limitations of RAG systems. *arXiv preprint arXiv:2509.18869*.

Shirley Wu, Parth Sarthi, Shiyu Zhao, Aaron Lee, Herumb Shandilya, Adrian Mladenic Grobelnik, Nurendra Choudhary, Eddie Huang, Karthik Subbian, Linjun Zhang, Diyi Yang, James Zou, and Jure Leskovec. 2026. Optimas: Optimizing compound AI systems with globally aligned local rewards. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

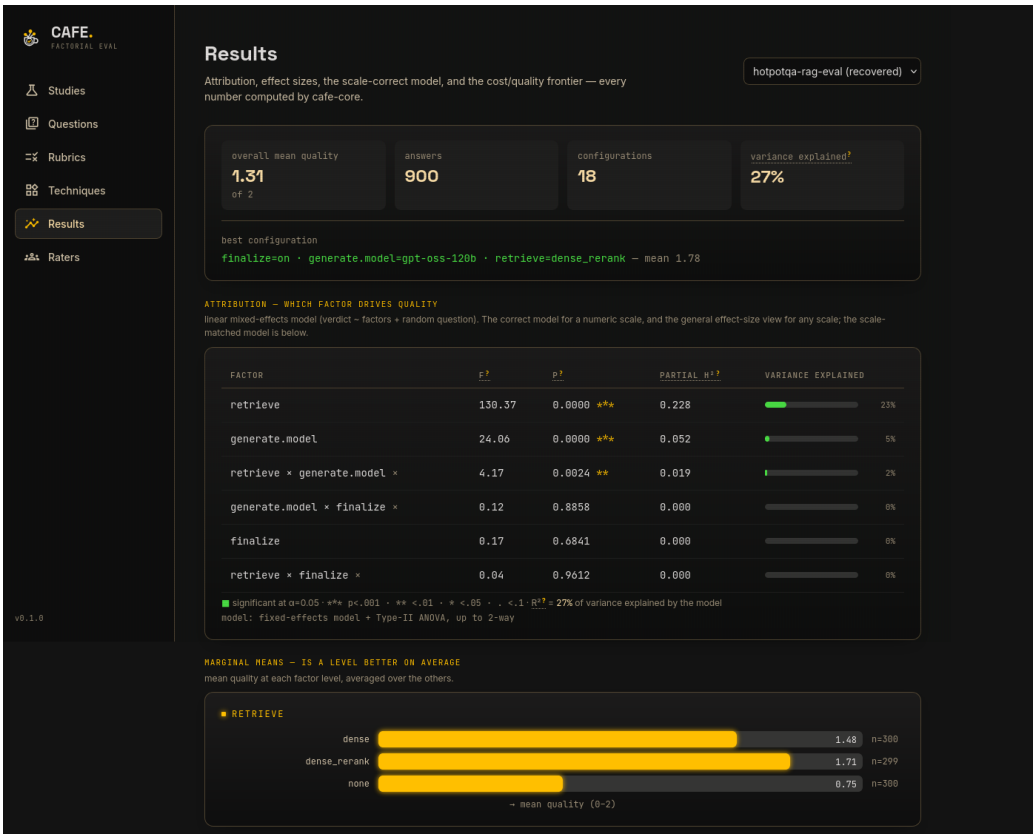
Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639:609–616.

Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research (BAIR) Blog.

A Supplemental Material

Figure 2: The CAFE web application (Results view) on the HotpotQA recovery study of Section 4: the factors `retrieve` × `generate.model` × the `finalize` placebo (18 configurations, 50 questions, a 0–2 correctness rubric). It reports overall quality, the best configuration, per-factor attribution (F, p, partial η^2, and the share of variance explained), and per-level marginal means, all computed by `cafe-core` and served over the API. Listing 2 is the same analysis as console output.



Listing 2: Complete console output of `result.report()` on the study of Figure 2: descriptive means (overall, ranked per configuration, and per factor) with the best configuration; the inferential linear mixed model (Type-II F, partial η^2, and pairwise Cohen's d) with its near-singular random-effect note; and the ordinal cumulative-link mixed model. Retrieval dominates and is correctly ordered (none < dense < dense_rerank), the base model helps, and the placebo `finalize` is null: the recovery result of Section 4.

```
900 answers | 18 configs | 50 inputs | 900 ratings
best: finalize=on | generate.model=gpt-oss-120b | retrieve=dense_rerank
pipeline: 900 answers -> 900 judged (1 unparseable) -> 899 usable verdicts

============================================================
DESCRIPTIVE -- means & best configuration
============================================================
verdicts: 900 (899 usable)   factors: retrieve, generate.model, finalize
overall mean quality: 1.31  (n=899)

per-configuration mean quality:
  1.78  (n=50)  finalize=on  | generate.model=gpt-oss-120b | retrieve=dense_rerank
  1.78  (n=49)  finalize=off | generate.model=gpt-oss-120b | retrieve=dense_rerank
  1.76  (n=50)  finalize=on  | generate.model=gpt-oss-20b  | retrieve=dense_rerank
  1.74  (n=50)  finalize=off | generate.model=gpt-oss-20b  | retrieve=dense_rerank
  1.62  (n=50)  finalize=off | generate.model=gpt-oss-120b | retrieve=dense
  1.60  (n=50)  finalize=on  | generate.model=gpt-oss-120b | retrieve=dense
  1.60  (n=50)  finalize=on  | generate.model=gemma3-4b    | retrieve=dense_rerank
  1.58  (n=50)  finalize=on  | generate.model=gpt-oss-20b  | retrieve=dense
  1.58  (n=50)  finalize=off | generate.model=gemma3-4b    | retrieve=dense_rerank
  1.50  (n=50)  finalize=off | generate.model=gpt-oss-20b  | retrieve=dense
  1.32  (n=50)  finalize=off | generate.model=gemma3-4b    | retrieve=dense
  1.28  (n=50)  finalize=on  | generate.model=gemma3-4b    | retrieve=dense
```

```
  1.16  (n=50)  finalize=on  | generate.model=gpt-oss-120b | retrieve=none
  1.10  (n=50)  finalize=off | generate.model=gpt-oss-120b | retrieve=none
  0.80  (n=50)  finalize=on  | generate.model=gpt-oss-20b  | retrieve=none
  0.74  (n=50)  finalize=off | generate.model=gpt-oss-20b  | retrieve=none
  0.36  (n=50)  finalize=off | generate.model=gemma3-4b    | retrieve=none
  0.36  (n=50)  finalize=on  | generate.model=gemma3-4b    | retrieve=none

per-factor marginal means:
  retrieve:        none 0.75 (n=300)   dense 1.48 (n=300)   dense_rerank 1.71 (n
     =299)
  generate.model:  gemma3-4b 1.08 (n=300)   gpt-oss-20b 1.35 (n=300)   gpt-oss-120b
      1.51 (n=299)
  finalize:        off 1.30 (n=449)   on 1.32 (n=450)

best configuration: finalize=on | generate.model=gpt-oss-120b | retrieve=
    dense_rerank  (mean 1.78)

============================================================
INFERENTIAL -- mixed-effects model (significance & effect sizes)
============================================================
verdict ~ (1 | input_id) + (retrieve + generate.model + finalize)^2
  fixed-effects model + Type-II ANOVA, up to 2-way   (n=899, alpha=0.05)

per-term effects  (F-test, p, partial eta^2;  'x' = interaction):
  term                            F          p    partial eta^2
  retrieve                   130.37     0.0000        0.228   ***
  generate.model              24.06     0.0000        0.052   ***
  retrieve x generate.model    4.17     0.0024        0.019   **
  generate.model x finalize    0.12     0.8858        0.000
  finalize                     0.17     0.6841        0.000
  retrieve x finalize          0.04     0.9612        0.000
  Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

effect sizes -- Cohen's d (magnitude of the gap; 0.2 small, 0.5 medium, 0.8 large):
  retrieve: dense vs dense_rerank              d = -0.31   95% CI [-0.47, -0.15]
  retrieve: dense vs none                      d = +0.86   95% CI [+0.69, +1.03]
  retrieve: dense_rerank vs none               d = +1.24   95% CI [+1.07, +1.42]
  generate.model: gemma3-4b vs gpt-oss-120b    d = -0.49   95% CI [-0.65, -0.33]
  generate.model: gemma3-4b vs gpt-oss-20b     d = -0.30   95% CI [-0.46, -0.14]
  generate.model: gpt-oss-120b vs gpt-oss-20b  d = +0.18   95% CI [+0.02, +0.34]
  finalize: off vs on                          d = -0.02   95% CI [-0.16, +0.11]

note: random intercept not estimable; using fixed-effects ANOVA
note: model was near-singular (little between-question variance) -- treat p-values
    as
      unstable; the effect sizes (Cohen's d) are the more reliable signal here

============================================================
ORDINAL -- cumulative-link mixed model (verdicts as ordered categories)
============================================================
ordinal CLMM -- verdict ~ (1 | input_id) + (retrieve + generate.model + finalize)^2
  (n=899, logLik=-557.7, alpha=0.05)

fixed effects (ordinal log-odds of a higher score; + = better):
  term                                                  estimate       p
  retrieve=dense_rerank                                   +1.149    0.0075   **
  retrieve=none                                           -3.944    0.0000   ***
  generate.model=gpt-oss-120b                             +1.250    0.0035   **
  generate.model=gpt-oss-20b                              +0.800    0.0541   .
  finalize=on                                             -0.070    0.8524
  retrieve=dense_rerank x generate.model=gpt-oss-120b     -0.146    0.7947
  retrieve=none x generate.model=gpt-oss-120b             +1.854    0.0006   ***
  retrieve=dense_rerank x generate.model=gpt-oss-20b      -0.041    0.9390
  retrieve=none x generate.model=gpt-oss-20b              +0.880    0.0936   .
  retrieve=dense_rerank x finalize=on                     +0.033    0.9418
  retrieve=none x finalize=on                             +0.089    0.8313
  generate.model=gpt-oss-120b x finalize=on               +0.160    0.7166
  generate.model=gpt-oss-20b x finalize=on                +0.302    0.4852
  Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```